

Table of Contents

1. Configuration Reference	1
1.1. Configuration Basics	1
1.2. Database Configuration	1
1.3. Authentication Configuration	2
1.4. Quantum Framework Configuration	4
1.5. See Also	6
1.6. Ontology Configuration (optional)	6
1.7. Query Gateway Configuration	7
1.8. Control-plane split (deployment mode)	8

1. Configuration Reference

This appendix explains key configuration properties used in Quantum applications, organized by functional area.

1.1. Configuration Basics

File Locations

- `src/main/resources/application.properties` - main configuration
- `src/test/resources/application.properties` - test overrides
- `.env` file - local development secrets (not committed)

Profiles and Environment Variables

```
# Profile-specific properties
%dev.auth.jwt.duration=7200
%test.quarkus.mongodb.database=test-db
%prod.quarkus.log.level=WARN

# Environment variable substitution
quarkus.mongodb.connection-
string=${MONGODB_CONNECTION_STRING:mongodb://localhost:27017}
auth.jwt.secret=${JWT_SECRET:change-me-in-production}
```

Precedence

System properties > Environment variables > `application.properties`

1.2. Database Configuration

MongoDB Connection

```
# Basic connection
quarkus.mongodb.connection-string=mongodb://localhost:27017
quarkus.mongodb.database=my-app

# With authentication
quarkus.mongodb.connection-
string=mongodb://user:pass@host:27017/database?authSource=admin

# Disable dev services (use real MongoDB)
quarkus.mongodb.devservices.enabled=false
```

Morphia Configuration

```
# Database and package scanning
quarkus.morphia.database=my-app-db
quarkus.morphia.packages=com.example.model,com.example.entities

# Schema management
quarkus.morphia.create-caps=true
quarkus.morphia.create-indexes=true
quarkus.morphia.create-validators=true
```

1.3. Authentication Configuration

JWT Provider

```
# Enable JWT authentication
quarkus.smallrye-jwt.enabled=true
auth.provider=custom

# JWT validation
mp.jwt.verify.publickey.location=publicKey.pem
mp.jwt.verify.issuer=my-app
mp.jwt.verify.audiences=my-app-users

# Custom JWT settings
auth.jwt.secret=${JWT_SECRET:your-secret-here}
auth.jwt.expiration=60
auth.jwt.refresh-expiration=1440
```

OIDC Provider

```
# Enable OIDC (alongside or instead of built-in JWT)
quarkus.oidc.enabled=true
auth.provider=custom,oidc          # or just: auth.provider=oidc

# OIDC configuration
quarkus.oidc.auth-server-url=https://your-provider.com/auth/realms/your-realm
quarkus.oidc.client-id=your-client-id
quarkus.oidc.credentials.secret=your-client-secret
quarkus.oidc.token.issuer=https://your-provider.com/auth/realms/your-realm

# Quantum OIDC provider settings
auth.oidc.provider-name=oidc          # Provider name (default: oidc)
auth.oidc.role-claim-path=groups      # Claim path for roles (default:
groups)
auth.oidc.username-claim=preferred_username # Claim for username (default:
preferred_username)
```

Dedicated Cognito Provider

```
# Enable Cognito provider (full AWS SDK integration)
auth.provider=cognito

# AWS Cognito settings
aws.cognito.user-pool-id=${USER_POOL_ID}
aws.cognito.client-id=${APPLICATION_CLIENT_ID}
aws.cognito.region=us-east-1
aws.cognito.jwks.url=https://cognito-
idp.${aws.cognito.region}.amazonaws.com/${aws.cognito.user-pool-id}/.well-
known/jwks.json
```

See [Multiple Authentication Providers](#) for details on configuring external identity providers.

Application Registration (optional module)

The `quantum-application-registration` module lets an application announce itself to its environment at startup, so first arrival can install, seed and migrate it. It is inert unless enabled, and requires an `ApplicationRegistrar` implementation on the classpath.

```
# Off by default; the framework starts identically without this.
quantum.application-registration.enabled=false

# Selects the registrar: matches its getName() OR its fully-qualified class name.
quantum.application-registration.provider=system-manager

# Required when enabled. Never inferred -- a service that guesses its
# environment can register into the wrong one.
quantum.application-registration.application-id=my-app
quantum.application-registration.environment=production

# Realms this application reads and writes, comma separated.
quantum.application-registration.realms=my-app-P1

# Build identity; participates in the already-registered fingerprint.
quantum.application-registration.application-version=1.4.2

# Whether a failed registration stops the service. Defaults to true.
quantum.application-registration.fail-on-error=true
```

Registration is attempted once per fingerprint (application + version + environment + realms). On every later boot the registrar's cheap local check short-circuits it, so a restart costs one indexed read rather than a call to the control plane.

See [Application Registration](#) for the SPI and implementation rules.

OAuth Server (optional module)

The `quantum-oauth-server` module uses the same JWT configuration properties listed above (`mp.jwt.verify.issuer`, `mp.jwt.verify.publickey.location`, `com.b2bi.jwt.duration`). No additional properties are required. The OAuth endpoints (`/oauth/authorize`, `/oauth/token`, `/oauth/jwks`, `/.well-known/openid-configuration`, `/oauth/userinfo`) activate automatically when the module is on the classpath.

OAuth clients are registered in the `oauth_clients` MongoDB collection. Authorization codes are stored in `oauth_authorization_codes` with a 5-minute TTL index for automatic cleanup.

See [OAuth Server Module](#) for client registration and integration examples.

Email Templates (Postmark)

```
# Postmark API key (required for email delivery)
postmark.api-key=${POSTMARK_API_KEY}

# Default sender address (required for email delivery)
postmark.default-from-email-address=noreply@yourdomain.com
```

If either property is missing, the `PostmarkEmailSender` logs an info message and skips delivery, allowing development environments to run without Postmark credentials.

Classpath templates are loaded from `templates/email/{templateKey}/` with files `subject.txt`, `body.html`, and `body.txt`. See [Email Templates](#) for details.

Shared Link Signing

```
# HMAC-SHA256 secret for signing share link tokens (required for link signing)
share.link.signing.secret=${SHARE_LINK_SECRET}
```

If the secret is not configured, `LinkSigner.isEnabled()` returns `false` and link signing/verification is disabled. See [Shared Links](#) for details.

1.4. Quantum Framework Configuration

Database Migration

```
# Migration settings
quantum.database.version=1.0.0
quantum.database.scope=DEV
quantum.database.migration.enabled=true
quantum.database.migration.changeset.package=com.example.migrations

# Optional CSV list of additional realms to initialize on startup before seed packs
run
```

```
quantum.migration.apply.realms=system-com,mycompany-com,test-com
```

Seed packs (filesystem + classpath)

```
# Enable seed pack discovery and application on startup
quantum.seed-pack.enabled=true
quantum.seed-pack.apply.on-startup=true

# Filesystem root for seed packs when present in the app repo
# If the path does not exist, the framework still loads classpath seed packs
# contributed by dependencies (e.g., quantum-framework) from resources under seed-
# packs/
quantum.seed-pack.root=src/main/resources/seed-packs

# Optional CSV filter to apply only specific seed pack names
# Example: quantum.seed.apply.filter=system-initialize,my-tenant
quantum.seed.apply.filter=
```



Classpath seed loading: The framework automatically discovers seed packs packaged under `seed-packs/` on the application classpath (including JAR dependencies). Apps depending on the framework no longer need to copy framework seeds locally or reference absolute file paths. If both filesystem and classpath contain seed packs with the same name, the latest semantic version is applied.

Bootstrap packs

```
# Enable BootstrapPack infrastructure
quantum.bootstrap-pack.enabled=true

# Apply discovered bootstrap packs on startup
quantum.bootstrap-pack.apply.on-startup=false

# Optional CSV list of realms to target on startup
# If omitted, the framework uses the system/default/test realms when available
quantum.bootstrap-pack.apply.realms=system-com,mycompany-com,test-com

# Optional CSV filter to apply only selected pack refs
quantum.bootstrap-pack.apply.filter=pack-a,pack-b
```



Bootstrap packs are not the same thing as seed packs.

- Use seed packs for deterministic baseline data.
- Use bootstrap packs for idempotent operational setup steps.

See [Bootstrap Packs](#) for details.

Security rules diagnostics

```
# Elevate RuleContext auto-reload diagnostics from DEBUG to INFO when troubleshooting
quantum.securityrules.ruleContext.debugAutoReload=false
```

Tips:

- In unit tests that run inside Quarkus, prefer using a CDI-managed `RuleContext`. A helper is provided in tests: `RuleContextTestUtils.reload(<realm>)` to force a refresh of policies from the repository for a given realm.
- If you construct `RuleContext` manually (outside CDI), repository injection is not available, and auto-reload is skipped to avoid wiping programmatically added rules.

Realm and Tenant Configuration

```
# System realm configuration
quantum.realmConfig.systemTenantId=system
quantum.realmConfig.systemAccountId=system-account
quantum.realmConfig.systemOrgRefName=SYSTEM
quantum.realmConfig.systemUserId=system-user

# Default tenant settings
quantum.realmConfig.defaultTenantId=default
quantum.realmConfig.defaultAccountId=default-account
quantum.realmConfig.defaultOrgRefName=DEFAULT

# Anonymous user (use carefully)
quantum.anonymousUserId=anonymous
```

1.5. See Also

- [Quarkus Configuration Guide](#)
- [Authentication Configuration](#)

1.6. Ontology Configuration (optional)

Enablement

```
e2eq.ontology.enabled=false # default; set true to enable ontology wiring in your app
```

TBox Persistence

```
# Enable TBox persistence to MongoDB (default: true)
# When enabled, the ontology TBox (schema) is persisted to MongoDB for faster startup
```

```
# and version history. The TBox is automatically rebuilt if the YAML source changes.
quantum.ontology.tbox.persist=true

# Force rebuild of TBox even if persisted version exists (default: false)
# Useful for development or when you want to ensure fresh rebuild from sources
quantum.ontology.tbox.force-rebuild=false

# Package scanning for @OntologyClass and @OntologyProperty annotations
quantum.ontology.scan.packages=com.example.model,com.example.entities
```

Registry modes

- In-memory (recommended to start): build a TBox in code/resources and instantiate `OntologyRegistry.inMemory(tbox)`.
- Mongo-backed (advanced): load a TBox from a Mongo collection and cache it in memory for hot reload/versioning.
- Persisted TBox (recommended for production): TBox is persisted to MongoDB and loaded on startup for faster initialization.

Edges collection and indexes

- The MongoDB `edges` collection (default name configured via `ontology.mongo.collection.edges`) supports open edge qualification via compound indexes:
- `idx_domain_p_dst_props: { "dataDomain": 1, "p": 1, "dst": 1, "props": 1 }`—optimizes `hasEdge / srcIdsByDst` lookups.
- `idx_domain_src_p_props: { "dataDomain": 1, "src": 1, "p": 1, "props": 1 }`—optimizes `hasIncomingEdge / dstIdsBySrc` lookups.
- Inferred edges carry `inferred: true` with provenance and timestamp metadata for maintenance and TTL eviction.

Materialization hooks

- Use `OntologyMaterializer` to recompute inferred edges when entities change (sources and intermediates for your property chains).

1.7. Query Gateway Configuration

Property	Default	Description
<code>feature.queryGateway.execution.enabled</code>	<code>false</code>	Guards execution of <code>POST /api/query/find</code> for <code>AGGREGATION</code> -mode queries. When <code>false</code> , aggregation queries return HTTP 501 (<code>ExecutionDisabled</code>). Must be set to <code>true</code> to allow the gateway to execute governed MongoDB aggregation pipelines (with inner-pipeline <code>\$lookup</code> pushdown and multi-hop traversals).

Property	Default	Description
<code>feature.queryGateway.maxPageSize</code>	<code>1000</code>	Maximum page size allowed for <code>/api/query/find</code> requests.
<code>ontology.mongo.database</code>	<code>quantum</code>	Default database name for the ontology edges collection when not overridden by realm context.
<code>ontology.mongo.collection.edges</code>	<code>edges</code>	Collection name storing ontology edge documents.

See also

- [Ontologies in Quantum](#)
- [Dynamic Edge Properties & Facet Segmentation](#)
- [Planner and Query Gateway](#)
- [User-Definable Facet Segmentation & Coordinate SPI](#)

1.8. Control-plane split (deployment mode)

Property	Default	Description
<code>quantum.mode</code>	<code>embedded</code>	Deployment mode of the control-plane split. <code>embedded</code> : the app owns its system realm — system-realm migrations, baseline identity, seeds, and bootstrap packs run locally (today's behavior, wp3 tiers 0/1). <code>remote</code> : the control plane owns the system realm; this app skips system-realm startup work while app-realm work runs locally (tier 2). Unknown values fail loud at startup.
<code>quantum.system-service.base-url</code>	—	Control-plane service endpoint. Required when <code>quantum.mode=remote</code> (startup fails loud without it); unused in embedded mode.
<code>quantum.system.directory.mode</code>	follows <code>quantum.mode</code>	Optional override for the SystemDirectory implementation alone (<code>local/remote</code>) — for migration windows where an app runs remote but still resolves the directory locally. Remote directory resolution fails loud until the Phase C control-plane client ships.

Property	Default	Description
<code>quantum.system-rest.enabled</code>	<code>true</code>	Build-time switch over the control-plane admin REST surface (<code>/admin/tenants*</code> , <code>/security/realm*</code> , <code>/security/realm-memberships</code> , <code>/admin/seeds</code> , <code>/system/migration</code> , <code>/admin/bootstrap-packs</code> , <code>/onboarding/*</code>). <code>false</code> removes all nine resources at build — the tier-1 hardening that replaces per-type <code>quarkus.arc.exclude-types</code> lists, and what a tier-2 tenant-plane app sets.